

Code Review & Judge's Audit

Streaming Live RAG — Samsung PRISM GenAI Hackathon 2026–27, Theme 4

Repository reviewed: Streaming-Live-Rag-main.zip -> streaming-live-rag/

Reviewed as of: 19 September 2026 (Day 3 of the 9-day build window; deadline 25 Sep, 11:59 PM)

Reviewer role: Senior developer code review + hackathon-judge compliance audit

Method: Full extraction and line-by-line reading of every file in the archive (106 files; 30 non-empty, 39 empty stubs, remainder are directory markers/bytecode cache), cross-checked against the Theme 4 guide's hard rules, pitfalls, and G1–G6 gates, and against current provider documentation for the two external model claims that could break the build.

1. Executive Summary

This is a **Day-3 checkpoint, not the final submission — and it should be read that way**. Judged strictly against the 9-day plan's own calendar, the team is slightly **ahead** of pace: Phase 2 (Hybrid Retrieval) was scheduled for today, and there is already a partially-working Phase 3 (Streaming Controller) wired in. That is a genuine positive and this report says so throughout.

That said, the review surfaced a meaningful number of **real, timeless bugs in the code that already exists** — not "not built yet" gaps, but places where the code is wrong, inconsistent with itself, or silently does the opposite of what its own comments and ADRs claim. Left alone, several of these will directly cost gate points on 25 September regardless of what gets added between now and then. The most serious:

1. The Retrieval Controller's decision is **computed and logged but never actually enforced** — the "no retrieval needed" (chit-chat / presentation-only) path still runs the full retrieval + LLM pipeline anyway, by the code's own admission in a comment.
2. A **telemetry timestamp bug** makes it currently impossible to compute G2 (early retrieval) from the logs the system produces, because the wrong clock is being recorded.
3. **Three different places in the codebase disagree** about which "fast" LLM model to call, and the team's own `.env.example` references a Gemini model that does not appear to exist yet.`
4. `docker-compose.yml` only starts the database, not the application — there is no Dockerfile and no one-command way to run the actual system, which is a direct risk to G1 (Reproducibility), the one gate that is graded strictly pass/fail.`
5. The project's own **Architecture Brief describes components (decomposition, session memory, grounding validation) as if they exist**; none of the eight files that would implement them contain any code yet.

None of this is unusual for a Day-3 hackathon build — it's exactly what a code review is for. Section 11 gives a concrete, priority-ordered list of what to fix first with 6 days remaining.

2. Scope & What Was Actually Checked

The archive contains **106 files**. Breaking that down honestly matters, because it is itself a finding:

Category	Count	Detail
Files with real content	30	Listed individually in Section 4 onward
Empty (0-byte) stub files	**39**	Every file under <code>backend/` (26/26), every file under <code>frontend/` (7/7), all six <code>eval/gates/g1..g6_*.py` files, and <code>eval/report.py`</code></code></code></code>
Directory markers / bytecode cache	37	<code>__pycache__/*.pyc` and empty directory entries</code>

The backend/ tree is a complete, empty duplicate of a working architecture — it mirrors (almost file-for-file) the retrieval/pipeline/telemetry/session structure recommended in the project's own execution plan, but not one file in it has a single line of code. Meanwhile, real, working code for a *subset* of the same functionality exists in a separate, flatter set of top-level folders (api/, controller/, retrieval/, streaming/, telemetry/). **Recommendation: delete the empty backend/ tree or clearly mark it NOT IN USE before submission** — as it stands, a judge opening the repo will reasonably wonder which of the two parallel structures is real, and an empty 26-file directory tree reads as either abandoned work or an unresolved merge between two team members who started in different places. Given this is exactly the kind of coordination gap flagged as a risk in the original execution plan, it's worth a 10-minute cleanup now rather than a confusing five minutes in front of the jury on demo day.

3. Progress vs. the 9-Day Plan (Credit Where Due)

Cross-referencing docs/RUNBOOK.md, agents.md, and the actual code:

Plan phase	Scheduled day	Status found in repo
Phase 0 — Research & Architecture Lock	Day 1 (17 Sep)	Done. 5 ADRs present and genuinely evidence-based (see §9).
Phase 1 — Corpus & Baseline Retrieval	Day 2 (18 Sep)	Done. <code>/turn`</code> endpoint answers queries end-to-end against the dev corpus.
Phase 2 — Hybrid Retrieval & Rerank	Day 3 (19 Sep, today)	Done. Dense+sparse RRF fusion and MiniLM cross-encoder reranking are both wired into <code>/turn`</code> .
Phase 3 — Streaming Controller (-> G2)	Day 4 (20 Sep)	Partially done, one day early. A real heuristics+LLM controller exists and is wired in — but see Critical Findings #1–#3, which mean it isn't yet doing what it's supposed to do.
Phase 4 — Decomposition (-> G3)	Day 5 (21 Sep)	Not started (on schedule).
Phase 5 — Synthesis, Grounding, Session (-> G4, G5)	Day 6 (22 Sep)	Not started (on schedule) — but the Architecture Brief already describes it as built (see §10).
Phase 6 — Telemetry, Repro, Demo UI (-> G1, G6)	Day 7 (23 Sep)	Not started (on schedule) — <code>docker-compose.yml`</code> 's app-container gap should be treated as a Day-7 priority, not a current failure.
Phase 7 — Hardening & Ablations	Day 8 (24 Sep)	Not started (on schedule). <code>BENCHMARK_REPORT.md`</code> is a template with every field reading "TBD" — expected at this point, but it's a large amount of work still ahead.
Phase 8 — Submission	Day 9 (25 Sep)	Not started (on schedule).

Net assessment: on pace, arguably slightly ahead on breadth. The risk isn't the calendar — it's that several of the "done" phases contain bugs that will quietly fail gates even once the remaining phases are built on top of them, because later phases depend on the data these components produce.

4. Critical Findings — Bugs in Code That Already Exists

These are not "missing features." This is code that runs today and does something other than what it is supposed to, or other than what its own comments/ADRs claim.

4.1 ■ Controller decisions are logged but never enforced

File: api/main.py, inside `handle_turn()`

```

elif decision_dict["trigger"] == "no_retrieval_needed":
# Fast-path for greetings etc.
# For this Phase, we'll just break and still execute the rest to see what happens, or skip Qdrant.
final_query = chunk.partial_text
break

```

The comment says the intent is to "skip Qdrant," but the code does not skip anything — execution falls straight through to the Qdrant hybrid search, reranking, and Gemini call regardless of what the controller decided. This is precisely the guide's **Pitfall #4 ("Ignoring Presentation-Only Turns")**, and it is not defused despite docs/ARCHITECTURE_BRIEF.md and the ADRs describing the controller as gating retrieval. Today's labeled set even contains chit-chat cases (q05, q06) that are meant to test exactly this path.

Fix:

```

elif decision_dict["trigger"] == "no_retrieval_needed":
    controller_decision_log = ControllerDecision(
        trigger="no_retrieval_needed", timestamp_s=chunk.t_offset_s,
        reason=decision_dict.get("reason", ""),
    )
    return TurnResponse(
        answer="<reformat/answer from session context, no corpus lookup>",
        telemetry=TelemetryEvent(
            session_id=req.session_id, turn_id=req.turn_id,
            controller_decision=controller_decision_log,
            retrieval_events=[], sub_queries=[], citations=[],
        ),
    )

```

Return early. Do not fall through to the retrieval block at all — this is the single highest-value fix in the whole review, because every chit-chat/presentation-only case in the eval set is currently silently mis-scored as "retrieval happened, and it was fine" rather than testing what it's meant to test.

4.2 ■ Retrieval-event telemetry logs the wrong clock — G2 is currently unmeasurable from this data

File: api/main.py

```

end_time = time.time()
...
retrieval_events=[
    RetrievalEvent(
        timestamp_s=round(end_time, 2), # <-- wall-clock epoch time
        query=final_query,
        trigger=controller_decision_log.trigger if controller_decision_log else "baseline_turn"
    )
],

```

`end_time` is `time.time()` — an absolute Unix timestamp (something like 1.79e9), captured *after* the LLM call finishes. Compare this to `ControllerDecision.timestamp_s=chunk.t_offset_s` a few lines earlier in the same function, which correctly uses the *simulated, utterance-relative* offset (e.g. 0.8). G2 ("retrieval commences prior to final transcript completion") is defined entirely in terms of the utterance's own timeline — with the current code, there is no way to tell from a `RetrievalEvent` whether retrieval happened at word 2 or word 20 of the utterance, because the value stored has nothing to do with the utterance at all.

Fix: reuse the same variable already computed for the controller decision:

```

retrieval_events=[
    RetrievalEvent(
        timestamp_s=controller_decision_log.timestamp_s if controller_decision_log else 0.0,
        query=final_query,
        trigger=controller_decision_log.trigger if controller_decision_log else "baseline_turn",
    )
],

```

4.3 ■ Early "provisional" retrieval never gets refined — the partial fragment becomes the permanent query

When `retrieve_now` fires on an early, partial chunk, `final_query` is set to that **partial fragment** and the loop breaks. There is no subsequent step that re-embeds or re-queries once the rest of the utterance arrives. In the guide's own worked example, an early "provisional" retrieve is explicitly the **start** of a process that keeps listening and refines/adds sub-queries as the utterance completes — here it is the entire process. This means the better the controller does its job (firing early), the more likely the final answer is based on an incomplete question. Recommend: on `retrieve_now`, keep listening to the end of the (simulated) utterance, treat the early hit as `trigger: "provisional"` telemetry, and re-run retrieval once more against the full utterance before synthesis — exactly as modeled in `docs/ARCHITECTURE_BRIEF.md`'s own example JSON, which already shows a "provisional" event followed by later events.

4.4 ■ No real streaming — "Time-to-First-Token" is measuring full-generation latency

The theme is graded partly on time-to-first-token, and the endpoint's own variable is named `ttft`. But:

- ``turn`` is a normal ``@app.post`` returning one JSON body — no ``StreamingResponse``, SSE, or WebSocket.
- The Gemini call (``gemini_client.models.generate_content(...)``) has no ``stream=True`` equivalent — it blocks until the full response is generated.

`ttft = (time.time() - llm_start) * 1000` is therefore total generation latency, not first-token latency. This will under-report the system favorably in dev testing and then read very differently against a genuinely streaming baseline. Recommend moving to Gemini's streaming generation call and a `StreamingResponse/WebSocket` endpoint before Phase 6 (this is squarely a Day 7 item per the plan, flagged here so it isn't a surprise).

4.5 ■ ``uncertainty`` and ``token_cost`` are defined in the schema but never populated

`telemetry/schema.py` correctly defines `uncertainty: str = ""` and a full `TokenCost` model — but `api/main.py`'s `TelemetryEvent(...)` construction never passes either. Both silently keep their zero-value defaults on every single turn, meaning: the system can never signal "the corpus didn't support this" (a hard requirement — "explicit uncertainty indicator... when the corpus lacks sufficient evidence"), and "cost per turn," an explicitly graded reporting metric, is currently always reported as `$0.00` regardless of actual spend. Both are one-line-per-field fixes once synthesis is built out in Phase 5, but worth flagging now since the schema plumbing is already there and unused.

4.6 ■ Citations are taken from what was **retrieved**, not from what the model **actually cited**

```
for hit in best_points:
    ...
    citations.append(tag)
```

`citations` is built from the chunks handed to the LLM, not parsed from the LLM's generated answer text. This means: if the model ignores a chunk it was given, that chunk is still reported as a citation; if the model hallucinates a `[Doc_ID §Section]` tag that wasn't actually retrieved, nothing catches it. ADR-5 explicitly specifies a "deterministic ID-membership validator" as the G4 mechanism — that validator does not exist in code yet. This is scheduled for Phase 5 (Day 6), so it isn't "late," but the current `citations` list should not be trusted as a G4 signal even informally until that validator exists.

4.7 ■ ``stream_simulator.py`` computes ``start_time`` and never uses it

```
start_time = time.time()
for i in range(0, len(words), words_per_chunk):
    ...
```

Dead variable — harmless on its own, but combined with `chunks = list(simulate_stream(...))` in `api/main.py` (which forces every chunk to be generated eagerly, before the loop that "processes" them even starts), there is no actual incremental/event-driven behavior anywhere in the pipeline: the "stream" is a precomputed list iterated synchronously. For the theme's purposes this is an acceptable simplification (the guide explicitly allows simulated input), but it's worth being honest in the Architecture Brief that "streaming" here means "replayed as pre-cut chunks with synthetic timestamps," not genuine incremental processing — otherwise a sharp judge will ask why an `asyncio`-based system (per ADR-1) has no `async def` or `await` anywhere in `api/main.py`.

5. Configuration & Dependency Issues

5.1 ■ Three different "fast model" defaults, disagreeing with each other

Location	Value
<code>`.env.example`</code>	<code>`FAST_LLM_MODEL=groq/compound-mini`</code>
<code>`controller/decide.py`</code> (hardcoded fallback)	<code>`"llama-3.1-8b-instant"`</code>
<code>`scripts/poc_b_llm.py`</code> (hardcoded fallback)	<code>`"groq/compound-mini"`</code>

agents.md's own log states plainly: `""The model*`llama3-8b-8192`*was decommissioned by Groq... updated*`FAST_LLM_MODEL`*to use*`groq/compound-mini`* — but that update never reached controller/decide.py, which is the file the running application actually imports. If FAST_LLM_MODEL is ever unset (e.g. a judge copies .env.example -> .env but a key gets dropped, or runs without dotenv loaded correctly), the live controller silently falls back to a value the team's own notes say they moved away from. groq/compound-mini itself checks out as a real, current Groq system as of this review. Fix:** set one value in one place — os.getenv("FAST_LLM_MODEL", "groq/compound-mini") in decide.py, matching the other two — and delete the hardcoded fallback in poc_b_llm.py in favor of importing the same constant.`

5.2 ■ ``.env.example``'s synthesis model likely does not exist; the code's own hardcoded fallback likely does

`.env.example` sets `SYNTHESIS_LLM_MODEL=gemini-3.8-pro`. Checking Google's current model documentation as of this review: there is a confirmed **Gemini 3.8 Flash** release, and the strongest current reasoning/"Pro"-tier model referenced is `gemini-3.1-pro-preview` (with even a "Gemini 3.5 Pro" reportedly still unreleased/unannounced with a firm date) — no "3.8 Pro" appears anywhere in current release notes or deprecation tables. If that holds, following the setup instructions in `.env.example` exactly would cause **every synthesis call to fail** with an invalid-model error, breaking the entire /turn endpoint. Ironically, `api/main.py`'s own hardcoded fallback (`"gemini-3.8-flash"`) does appear to be a real, current model — meaning the app is currently *more* likely to work with no `.env` file at all than with one built from the template you're shipping to judges. **Action: verify the exact current Pro-tier model ID at ai.google.dev before 25 September and fix `.env.example` to match** — this is a two-minute check that could otherwise cost the whole demo on judgment day if a judge's environment happens to pick up the example value.

5.3 ■ ``docker-compose.yml`` only runs the database — there is no way to boot "the system" in one command

```
services:
  qdrant:
    image: qdrant/qdrant:latest
    ...
```

That's the entire file. No `api` service, no Dockerfile anywhere in the repository. `docs/RUNBOOK.md` confirms this isn't an oversight in the compose file alone — the actual documented startup procedure is `docker compose up -d` (Qdrant only) **then** `source venv/bin/activate` **then** manually run `uvicorn api.main:app --reload`. That is a three-step manual process, not the single-command clean-machine boot G1 requires ("Container launches via a single command on a clean machine"). This is squarely a Phase 6 (Day 7) deliverable per the plan, so it is not late — but it is the single gate with zero partial credit, so it deserves to be first in the queue on Day 7, not last. A minimal fix is a two-line addition once a Dockerfile exists:

```
api:
  build: .
  ports: ["8000:8000"]
  env_file: .env
  depends_on: [qdrant]
```

5.4 ■ Undeclared dependency: ``pyyaml``

`eval/bench_controller.py` does `import yaml`, but `pyyaml` is not listed in `pyproject.toml`'s dependencies. On a genuinely clean environment where nothing transitively pulls it in, this raises `ModuleNotFoundError` before the benchmark can even run. Add `"pyyaml>=6.0"` to `pyproject.toml`.

5.5 ■ No pinned/locked versions

`pyproject.toml` uses `>=` minimum bounds only (`fastapi>=0.100.0`, `qdrant-client[fastembed]>=1.9.0`, etc.), with no lockfile (`uv.lock`, `poetry.lock`, or a frozen `requirements.txt`). A future breaking release of any dependency between now and 25 September could silently change runtime behavior on a fresh install. Recommend generating a lockfile once the dependency set stabilizes, per the plan's own reproducibility guidance.

5.6 ■ Minor hygiene

- No root-level `README.md` exists anywhere in the archive — only `docs/RUNBOOK.md`. The brief explicitly requires a README with setup instructions; even if `RUNBOOK.md` is kept as the detailed version, a short root `README.md` pointing to it (and to Docker) should exist.
- `.gitignore` contains a single line (`.env`) — no secrets were found committed, which is good — but it doesn't exclude `__pycache__`, `*.pyc`, or virtual-env directories, and compiled `.pyc` files were in fact present in the shared archive.
- `docker-compose.yml` uses the obsolete `version: '3.8'` key, which modern Docker Compose ignores with a deprecation warning — harmless, but worth removing.
- The 559 KB copy of the execution-plan PDF sits at the repository root rather than under `docs/`; purely cosmetic, but worth tidying before the tagged submission commit.

6. Evaluation Methodology Issues

The one benchmark script that exists (`eval/bench_controller.py`) has real methodology problems that will produce a number, but not a *meaningful* one:

Issue Detail	
Hardcoded domain proxy	"capacity" not in <code>utterance.lower()</code> is used to decide whether a query is "no-retrieval-eligible." This only works because the toy corpus happens to use that exact word — it will not generalize to the official corpus or to any rephrasing, and it edges uncomfortably close to the kind of query-specific pattern-matching the guide's "No Hardcoding" rule warns against (this is eval code, not the graded system itself, but it means the resulting G2 number is not trustworthy evidence of anything beyond this exact dev set).
Lenient "early" definition	"Eligible" = more than 2 simulated chunks; "triggered early" = anything before the <i>*last*</i> chunk. A system that only ever retrieves at 90% of the way through an utterance would score 100% "early" under this test, which doesn't reflect the guide's own worked example (retrieval starting at 0.8s into a 2.1s utterance — roughly the one-third mark).
No retrieval-correctness check at all	<code>expected_doc</code> is read from <code>labeled_set.yaml</code> but never compared against anything the system actually retrieves — the script only checks <i>*whether*</i> the controller fired, never <i>*whether firing led to the right document*</i> .
No persistence	Results print to stdout only; nothing is written to a file, so there's no audit trail once the terminal closes.
Missing scripts	<code>bench_retrieval.py</code> , <code>bench_decomposition.py</code> , <code>bench_grounding.py</code> , <code>bench_session.py</code> , and <code>validate_schema.py</code> (all named in the execution plan) do not exist yet — expected at this stage, flagged as a large remaining workload for Days 4–7.

Eval-set data quality: two of the fifteen labeled queries don't actually match their own ground truth. `q02` ("Who needs to approve domestic flights?") and `q10` ("...without VP approval if it's **domestic**?") are both labeled `expected_doc: Doc_02` — but `Doc_02`'s actual text only discusses **international** travel approval and VP sign-off for late-booking exceptions; it never mentions domestic flights. Even a perfect retrieval system has nothing correct to retrieve for these two labels as written. Recommend either rewriting the corpus to actually cover a domestic-flight rule, or rewriting the two queries to match what the corpus really says, before trusting any recall numbers built on this set.

7. Documentation vs. Reality — Handle Before Submission

- This is less a code bug than a submission-integrity issue worth fixing deliberately, because two of the documents involved are graded deliverables:
- `docs/ARCHITECTURE_BRIEF.md` (which maps directly to the required PPT content item "solution and architecture diagram") describes the Multi-Intent Decomposer, Session-Aware Synthesizer, Session Store, and Eval Harness with concrete function signatures, as though they are built. None of the corresponding files contain any code. This isn't dishonest — it's a plan document doing its job — but as written today it does not distinguish "built" from "designed," and a jury reading only the brief would reasonably assume more is finished than is. Recommend annotating each row of the Component Boundaries table with a status (■ Built / ■ In Progress / ■ Planned) before this document goes into the final PPT.
 - `docs/POC_RESULTS.md` marks POC A `***Status: Done***` directly above a line reading `***End-to-End Latency: Search step pending implementation***`. `scripts/poc_a_qdrant.py` itself contains an explicit comment block: `***NOTE: To complete this POC... you will need to: [generate embeddings, insert them, run hybrid search, measure latency]***` — none of which the script does. The collection gets created and nothing else happens. "Done" should read "Infra verified; search/latency not yet measured" — the honest version is one line away and costs nothing to fix now, before it's read by a judge instead of a teammate.
 - `docs/BENCHMARK_REPORT.md` is entirely template ("TBD" in every cell) and `docs/CHECKLIST.md` has zero of its 13 boxes checked — both expected at Day 3, both worth revisiting as literal, load-bearing to-do lists for Days 7–9 rather than documents to forget about.

8. Gate-by-Gate Risk Assessment

Gate Threshold	Current state	Risk if unaddressed	
G1 Reproducibility	Single-command clean-machine boot	No Dockerfile; compose file starts only Qdrant; manual <code>venv+uvicorn</code> steps documented	■ High — this gate is pass/fail with no partial credit
G2 Early Retrieval	≥80% eligible, low false-trigger	Controller exists but suppression path doesn't suppress (4.1) and the telemetry needed to score this is currently broken (4.2)	■ High until 4.1/4.2 are fixed — likely quick to resolve
G3 Multi-Intent	≥70% of compound queries	Not started (on schedule for Day 5)	■ On track
G4 Grounding	≥85% citation support, zero fabrication	Citations are retrieval-derived, not answer-derived; no validator yet (4.6)	■ On track, but note the current <code>`citations`</code> field shouldn't be trusted even informally yet
G5 Session Refinement	Verified continuity	No session store; <code>`SessionState`</code> defined but unused	■ On track for Day 6
G6 Telemetry	100% field coverage	Schema is complete; <code>`uncertainty`</code> and <code>`token_cost`</code> fields are wired but never populated (4.5); no on-disk persistence of logs yet	■ On track, straightforward once synthesis is built

9. What's Working Well

- A fair review credits this too:
- The ADRs are genuinely evidence-based, not guesswork. ADR-4 in particular documents a real Day-1 finding (Gemini alone measured at 5,000–16,000 ms, correctly rejected for the latency-critical controller role in favor of Groq) — that's exactly the kind of measured, defensible engineering decision a jury rewards under "Technical depth & feasibility."
 - Hybrid retrieval is real and correctly built, closely following ADR-2/ADR-3: Qdrant with named dense (``bge-small-en-v1.5``) and sparse (``Qdrant/bm25``) vectors, RRF fusion via a single ``prefetch`` query, MiniLM cross-encoder reranking — this is not a stub, it runs.

- **retrieval/ingest.py** is clean, idempotent (drops and recreates the collection on every run), and correctly tags chunks with `Doc_ID/Section` metadata for later citation.
- **telemetry/schema.py**'s Pydantic models closely match the guide's own worked JSON example — good foundation, just needs every field actually populated by the handlers that construct it.
- **No secrets committed**; placeholder-value checks in `poc_b_llm.py` (`if api_key == "your_groq_api_key_here"`) are a nice defensive touch most hackathon teams skip.
- **Retry/backoff logic** is present at both LLM call sites, not just hoped for.
- **Pace**: substantially ahead of the plan's own Day-3 target.

10. Prioritized Fix List for the Remaining 6 Days

Ordered by (a) how cheap the fix is and (b) how much gate-risk it removes:

#	Fix	Effort	Why it's high priority
1	Make the <code>no_retrieval_needed</code> path actually return early instead of falling through (§4.1)	~15 min	Currently the single biggest gap between "controller decides" and "controller controls"
2	Fix <code>RetrievalEvent.timestamp_s</code> to reuse <code>chunk.t_offset_s</code> (§4.2)	~2 min	Without this, G2 cannot be measured from your own telemetry at all
3	Reconcile the three conflicting <code>FAST_LLM_MODEL</code> defaults to one value (§5.1)	~10 min	Silent fallback to a value your own team already moved away from
4	Verify <code>SYNTHESIS_LLM_MODEL</code> 's real current ID against <code>ai.google.dev</code> and fix <code>.env.example</code> (§5.2)	~10 min	A wrong model ID breaks every synthesis call
5	Add <code>pyyaml</code> to <code>pyproject.toml</code> (§5.4)	~1 min	Clean-install failure otherwise
6	Delete or clearly label the empty <code>backend/</code> tree (§2)	~5 min	Removes a confusing, misleading artifact from the submitted repo
7	Correct <code>docs/POC_RESULTS.md</code> 's "Done" status for POC A (§7)	~2 min	Submission-integrity, costs nothing
8	Fix the two mislabeled eval queries or the corpus content behind them (§6)	~15 min	Otherwise your own G2/retrieval numbers rest on bad ground truth
9	Build the Dockerfile + <code>api</code> compose service (§5.3)	~1–2 hrs	Scheduled for Day 7 — flagged early because G1 is binary
10	Build the deterministic citation validator (§4.6) before trusting any G4 number	Scheduled Day 6	Needed before G4 numbers mean anything
11	Add real token-level streaming and a correct TTFT measurement (§4.4)	Scheduled Day 6–7	Currently mislabels total latency as first-token latency

11. If Judged Today (For Calibration Only — 6 Days Remain)

This is explicitly **not** a final grade — six days and five full plan-phases remain. It's included so the team can see where effort currently converts into score under the brief's actual weighting (Functionality 30% / Technical depth 25% / Innovation 20% / Relevance 15% / Presentation 10%):

- **Functionality (30%)**: Partial — the working slice (ingest -> hybrid search -> rerank -> cited answer) is real and demoable, but the suppression bug and streaming gap mean the "live, incremental" half of the theme isn't yet demonstrable.

- **Technical depth (25%):** Solid start — the ADRs and the hybrid-retrieval implementation show real engineering judgment; undermined right now by the config drift and unmeasurable G2 telemetry, both fixable in minutes.
- **Innovation (20%):** Not yet assessable — the theme's most novel asks (decomposition, session-aware refinement) are the unbuilt Phase 4–5 work.
- **Relevance to theme (15%):** On track — the architecture faithfully follows the guide's component model.
- **Presentation & documentation (10%):** Currently a mixed signal — the ADRs and plan documents are unusually thorough for Day 3, but the Architecture Brief overstating build status (§7) would read poorly to a judge comparing docs against code line by line, exactly as this review did.

Bottom line: nothing found here is unfixable, and most of it is cheap. The highest-leverage use of the next work session is items 1–5 in Section 10 — all under an hour combined — before adding any new feature surface.